

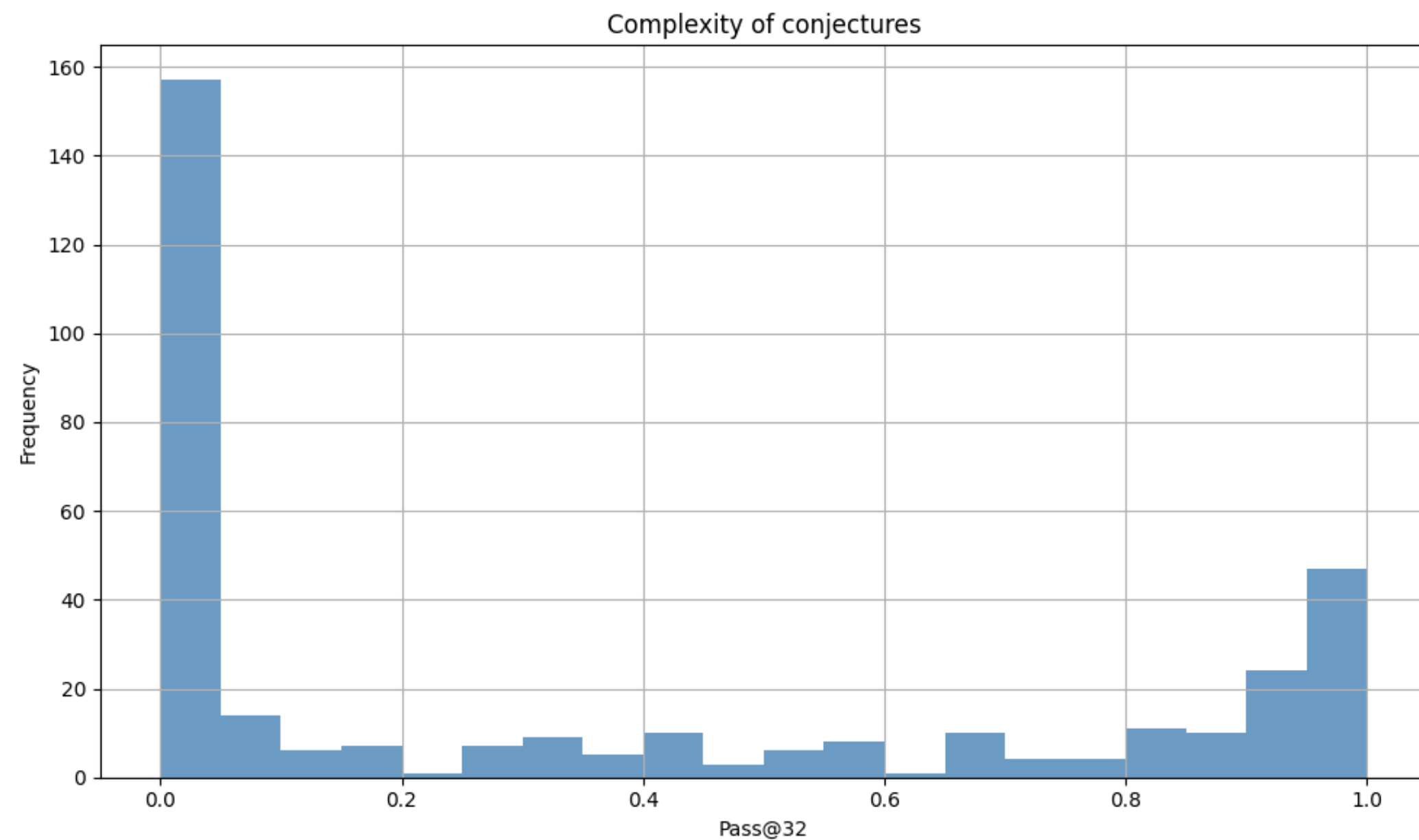
Reinforcement Learning for Conjecture Generation in Lean

Work done at Internship at Harvard

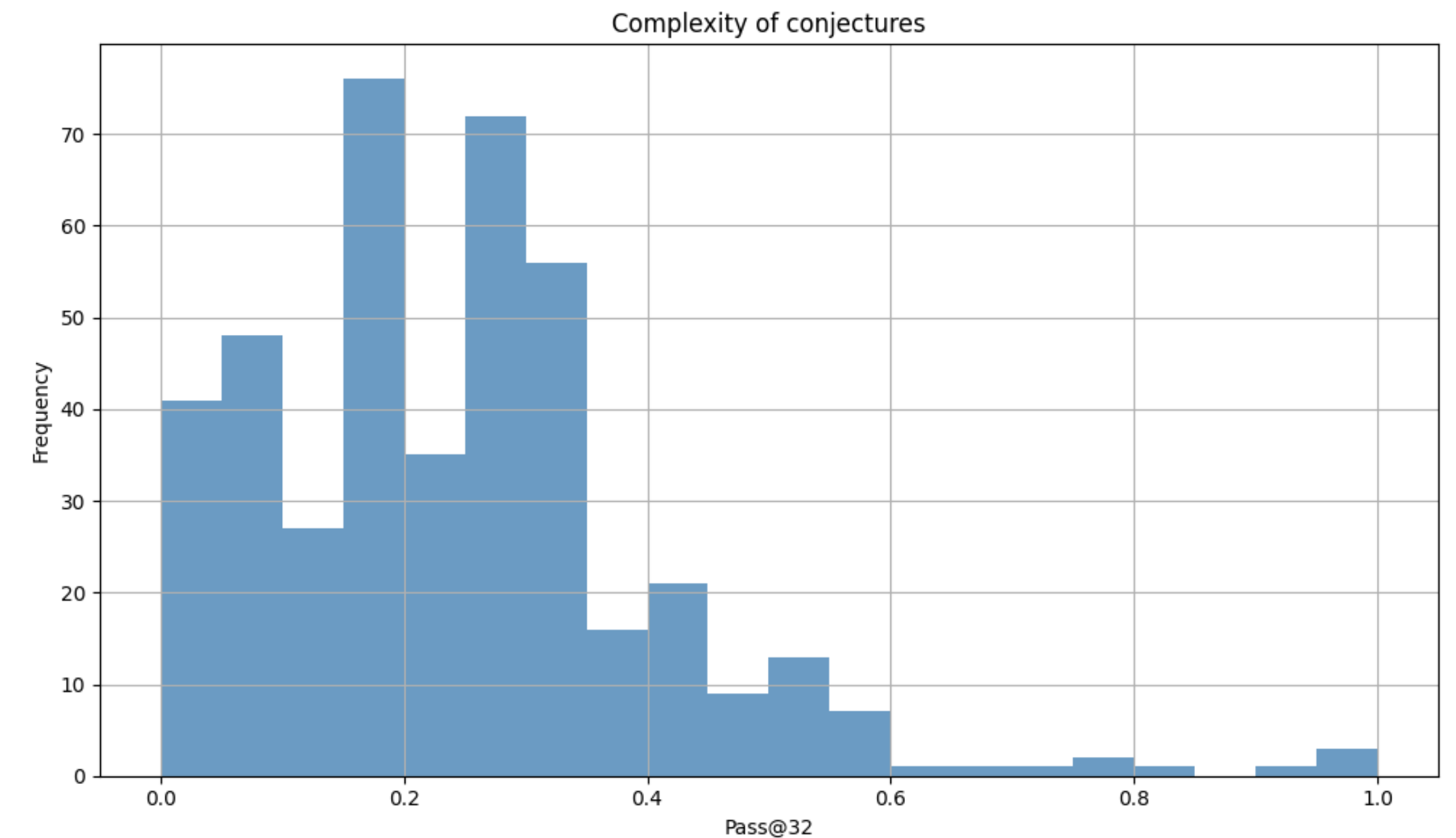
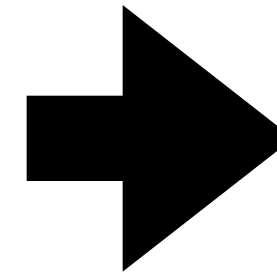
Motivation :

- No ground truth is needed when generating synthetic data in Lean.
- An under explored area: STP trains a conjecturer in Lean for the first time.
- Does the conjecturer deserves better training ?

Training the conjecture :



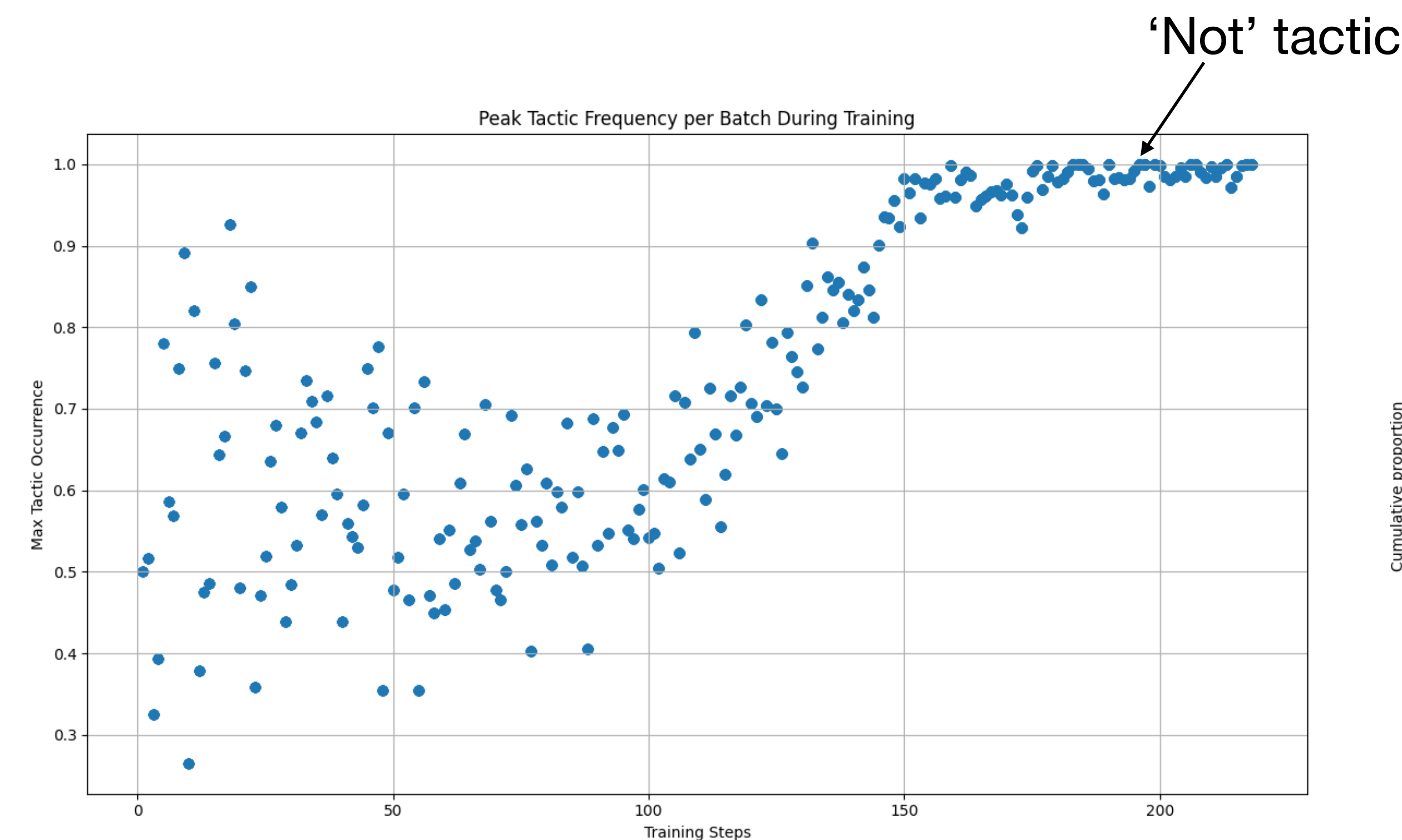
Before RL training



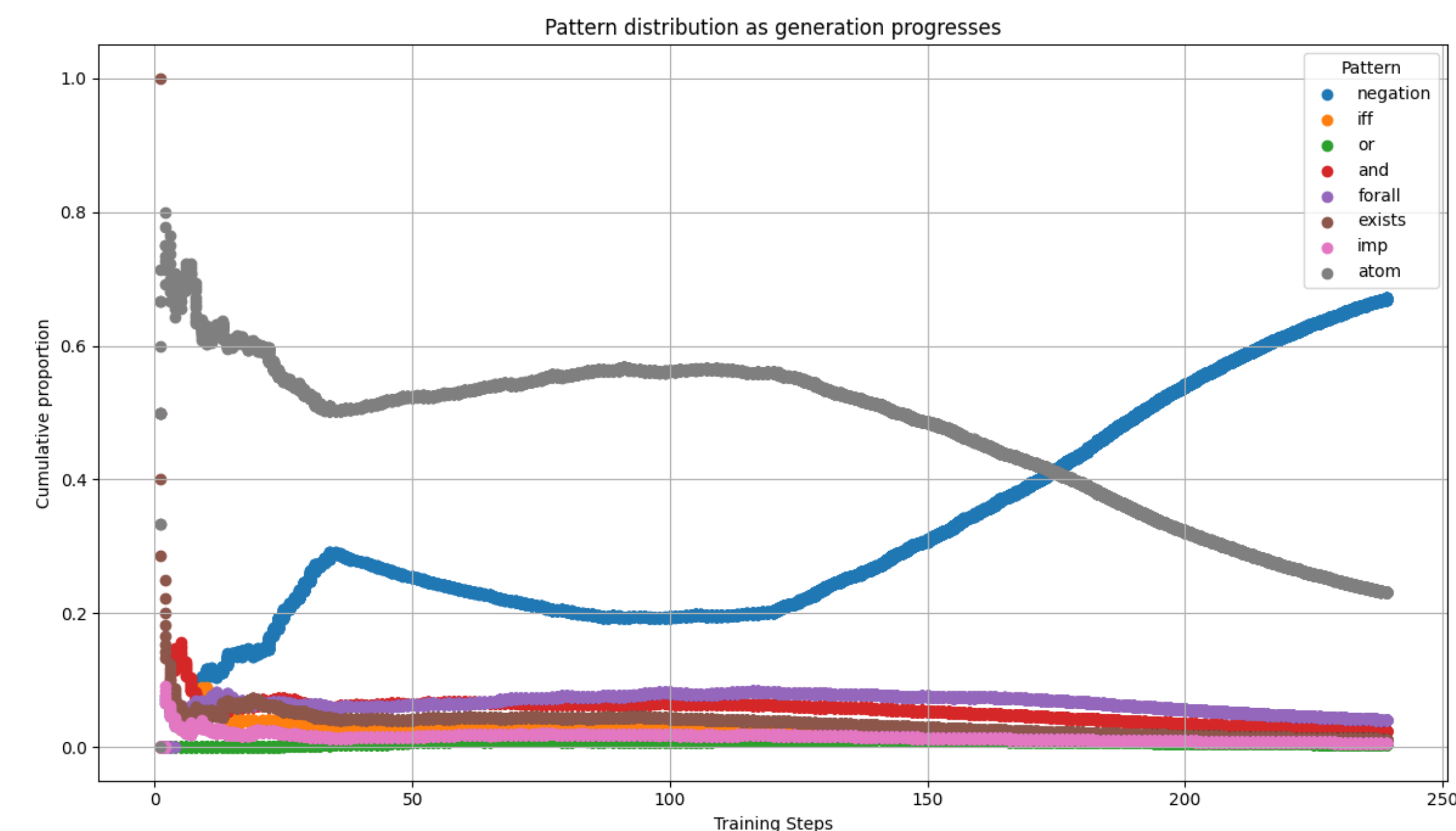
After RL training

=> After RL training, the conjecturer learns more about the complexity of the theorems it should generate.

Do not train the conjecture too long : Pattern Generation



(50,0.7) means at the 50 step of training there is a tactic that was used in 70% of the correct statements.



Full Leanworkbook data

Basic reward functions

Reward Definition	Comments
$\text{Pass@8} \in (0, 0.5]$	Model repeats the same statement; i.e., <code>old_statement = new_statement</code> .
$\text{Pass@8} \in (0, 0.5] \cdot \mathbb{1}[\text{new} \neq \text{old}]$	The LLM discovers a valid statement and keeps repeating it, ignoring changes in input.
$\text{Pass@8} \in (0, 0.5] \cdot \mathbb{1}[\text{new} \neq \text{old}] \cdot \mathbb{1}[\text{not generated before}]$	Aims for minimal changes to previously valid but unseen statements.
$\text{Pass@8} \in (0, 0.5] \cdot \cos(\text{new}, \text{old}) \in (0.4, 0.9)$	Model generates diverse yet similar theorems. Found 116/434 that meet the condition.

Importance of comparing to previous generated examples ==> Novelty

$$\text{Reward_base} = \underbrace{\text{Pass@8} \in (0, 0.5]}_{\text{Complexity}} \cdot \underbrace{\mathbb{1}[0.4 < \cos(\text{new}, \text{old}) < 0.9]}_{\text{Relatedness}} \cdot \underbrace{\mathbb{1}[\min \cos(\text{new}, \text{prev}) < 0.9] \cdot \mathbb{1}[\min \cos(\text{same batch}) < 0.9]}_{\text{Novelty}}$$

Pattern Generation

Find a pattern and use it to generate all the proofs

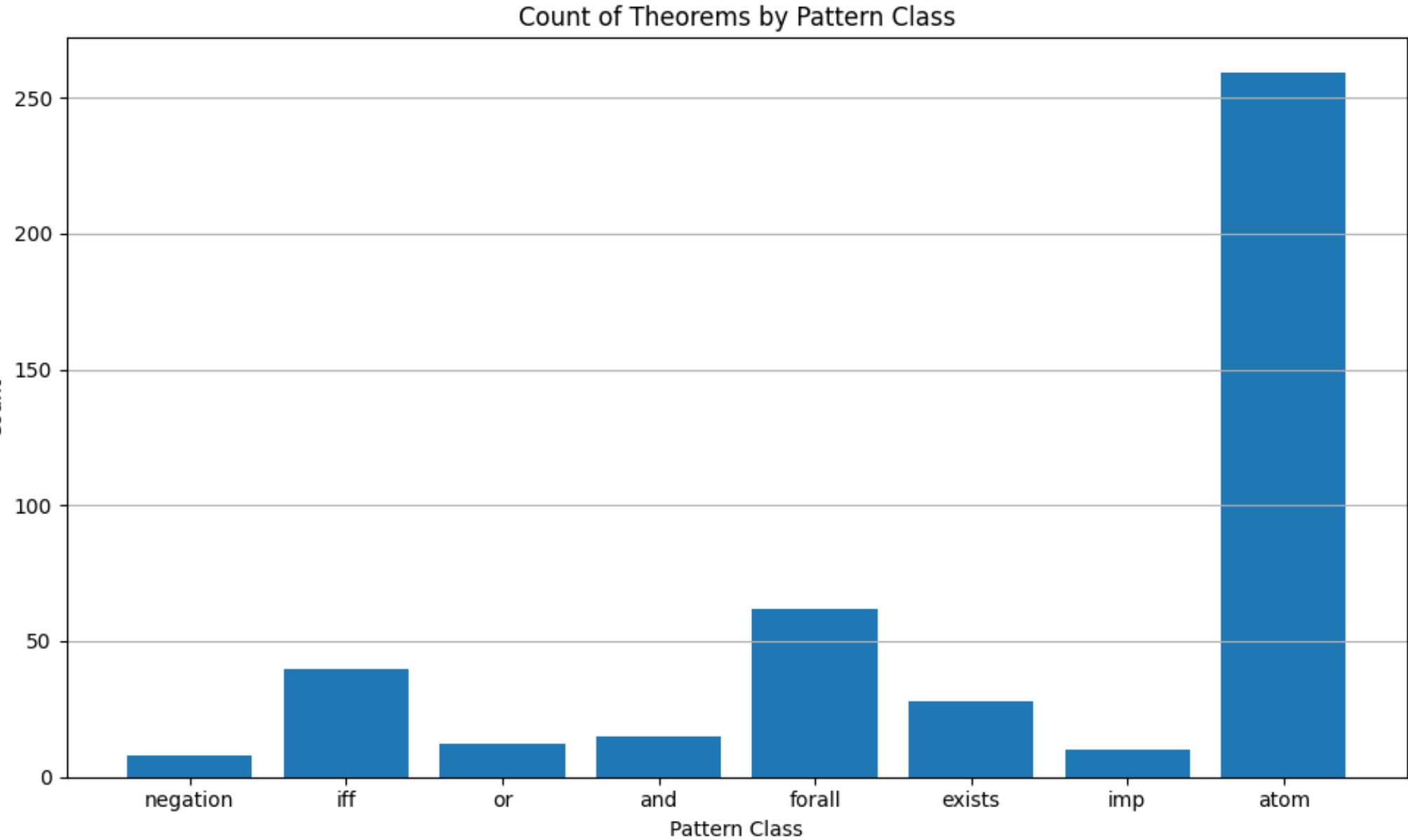
Reward Definition	Comments
Reward_base	Generate only statements seeking counterexamples, e.g., $\neg(\forall x : \dots)$.
Reward_base . 1[not a counterexample]	Generate only existential statements that need to prove existence, typically starting with $\exists$.
Reward_base with new SFT model	Generate only statements that contain the logical operator or .
Sum of Reward_base terms	Generate only statements that contain the logical operator And .

Can theorems be classified to ensure that no pattern dominates?

Patterns in Lean statements

Pattern	Symbol	Description
negation	$\neg$	The statement is a negation.
exists	$\exists$	The statement expresses an existential claim.
forall	$\forall$	The statement expresses a universal claim.
iff	$\leftrightarrow$	The statement is a biconditional (“if and only if”).
or	$\vee$	The statement is a disjunction.
and	$\wedge$	The statement is a conjunction.
imp	$\rightarrow$	The statement is an implication.
atom	—	The statement is atomic (none of the above apply).

Table 23: Syntactic patterns of Lean statements



Theorem Count per Pattern Class before RL